Praevisa — Technical Briefing for the CTO

Date: 2026-06-08 (rev. 2026-06-09) · **Purpose:** orient a new technical lead on where the prediction engine actually stands — what is validated, what is promising-but-unproven, what data we depend on, and what decisions are open. Written to be self-contained; no prior project context assumed.

> ***2026-06-09 revision.*** *Folded in two items from the* `part1-contested` *branch: a first measured competitive benchmark (§7) and a documentation-integrity note on a retired legacy figure (end of §2). A later same-day pass fixed a committee-vote subject-parsing bug and re-ran on the corrected, larger sample (§4): the* ***contested*** *Stage-A result moved from p = 0.062 (n = 6) to* ***p = 0.039 (n = 8)*** *— now significant — while one secondary cut (all-pairs vs* `baseline_A`*) softened to p = 0.057. This is the one validated result that changed; it is documented in place rather than silently updated.*

---

0. What Praevisa is

Praevisa forecasts how the **European Parliament (EP)** votes on legislation *before the vote happens*. Predictions are at the level of the eight-nine **political groups** (EPP, S&D, Renew, Greens, etc.), not individual MEPs and not just pass/fail. The intended buyers are public-affairs / lobbying / analyst teams who need to know which way a file is going and **who would have to change their vote to flip the outcome**.

1. 60-second primer on the process (the rest depends on it)

A legislative file under the **ordinary legislative procedure** (referenced as `YYYY/NNNN(COD)`) moves roughly:

> ***lead committee*** *drafts and votes a report -> **plenary** (the full Parliament) votes at first reading -> **trilogue** negotiations with the Council of member states -> final adoption.*

Two stages produce public, per-group **roll-call** data we can use: the **committee** vote and the **plenary** vote. Both matter below.

The single most important fact about this domain: the vast majority of final plenary votes pass, and are *trivially* predictable from party arithmetic (a "always predict pass" rule is ~90% accurate). All real forecasting value lives in the small **contested** subset — close votes, rejected files, and files where a major group breaks from its usual position. Any honest evaluation has to be judged *there*, not on the easy majority.

2. Why this work was necessary (a credibility reset)

Before this session, the engine's reported quality rested on accuracy figures that, on inspection, had **no reproducible artifact** behind them — you could not regenerate them from code and data. The first job was therefore not modeling but **governance**: make every number reproducible from committed code, and delete anything that isn't. A CTO should read the rest of this memo through that lens — the bar we now hold is "one command regenerates it, or it doesn't get claimed."

The discredited figure was a group MSE of **0.1004 / 0.1508**; the real, twice-reproduced numbers are **0.1656 / 0.3240** (§3a). The old figures are kept in the docs only as labeled tombstones so no one resurrects them. One subtlety worth flagging, since it protects this reset: Baseline A's *actual* computed loss on the routine subset is 0.15081, which coincidentally collides with the dead `0.1508` — `RESULTS.md` now carries an explicit note so no reader mistakes the real value for the discredited one.

3. What is now validated

3a. Reproducible baselines; unsubstantiated numbers removed

We rebuilt the baseline predictors as committed, one-command-reproducible code over a frozen 22-file test set of real EP votes (source: HowTheyVote.eu), and **independently re-derived the same numbers in a**

second repo with separate code and a fresh data pull.

Terms, briefly:

- **Baseline A** — predict each group's vote as its own historical average ("what this group usually does").
- **Baseline C** — a fixed rule: the centrist coalition votes yes, the flanks vote no.
- **Floor / "pooled mean"** — predict the single overall average for every group; the dumbest reasonable predictor, and the one a real model must beat.
- **MSE** — mean squared error between predicted and actual group yes-rate (lower = better). Evaluated leave-one-out (each file predicted without seeing itself).

metric	previously cited (no artifact)	reproduced (two independent code paths)
Baseline A group MSE	0.1004	**0.1656**
Baseline C group MSE	0.1508	**0.3240**

The previously cited numbers do not reproduce under any faithful build; the real ones reproduce exactly, twice. Method and decisions are frozen in `METHODOLOGY.md` / `RESULTS.md`.

3b. The hard finding: history-only baselines have no edge where it counts

Significance testing (cluster bootstrap, 10k resamples) shows **Baseline A is statistically indistinguishable from the pooled-mean floor** on the full set (p = 0.35). We then restricted evaluation to the **contested** subset — and it gets worse, not better: on no contested cut does Baseline A beat the floor, and on *rejected* files it is actively *worse* than the pooled mean.

Takeaway: "what a group usually does" is not a forecasting edge. This is a useful negative result — it tells us where *not* to invest — and it sets a hard, explicit bar: **a real model must beat the pooled-mean floor on the contested subset, prospectively.**

4. The first positive result — committee vote predicts plenary

Hypothesis: on contested files, a group's **committee-stage** vote (cast weeks before plenary, so legitimately a *pre-vote* signal) predicts how that group votes in plenary far better than any history-only baseline.

We built the data pipeline to test this (§5) and graded it time-split and pseudo-prospectively (the floors are fit only on data dated *before* each target; the committee vote precedes the plenary vote). Metric: per-group MSE, with paired bootstrap confidence intervals and a Wilcoxon test.

subset	n	committee signal MSE	floor MSE	95% CI of difference	p
all pairs (vs const_mean)	13	**0.069**	0.162	[-0.157, -0.031]	**0.027**
contested only (vs both floors)	8	**0.068**	~0.20	[-0.215, -0.036]	**0.039**

> ***2026-06-09 update — sample correction strengthens the contested result.*** *The signal selector was matching scraped-PDF subject lines against an exact-string table, which silently dropped legitimate lead-committee report votes carrying template noise (`1.1.`, bullets, a `- Rejected` suffix). Fixing that parser recovered **+6 graded pairs (+2 contested)** — including a file the committee and plenary both **rejected**, the most diagnostic contested case. On the corrected sample the **contested** result now reaches **p = 0.039 against both floors** — it crosses 0.05 for the first time (was p = 0.062 at n = 6). One honest cost: all-pairs-vs-`baseline_A` softened from p = 0.020 to **0.057** (the added files are ones `baseline_A` predicts well); it is reported, not hidden. Numbers below are the corrected sample.*

- On the **contested subset — the only cut that counts** — the committee signal beats both §9.6 floors with **p = 0.039**, winning 6 of 8 files including the rejected one. This is the first time the contested result is significant rather than only directional. (Pre-fix it was n = 6, p = 0.062.)
- **Biggest caveat, unchanged:** the contested files are *clustered* (shared committees; the ECON pair are near-twins), so the effective independent n is ~3-4, not 8 — the p understates how correlated the evidence is. Encouraging, not yet conclusive.
- **We stress-tested it.** A skeptic would say "committee and plenary are nearly the same vote." We split by stage: the genuinely-earlier **report-adoption** committee vote is the *strongest* predictor (0.052 vs 0.219 on contested report files), while the late, near-identical "provisional agreement" vote is *weaker* — the opposite of what an artifact would show. The signal is real.

This is the first evidence in the project that any pre-vote signal beats the established baseline on the votes that matter. It is not yet a validated track record (see §7).

5. Data sources and their reliability (important for a CTO)

- **HowTheyVote.eu** — our source for plenary roll-calls (per-group results). Two access paths: a JSON API and a nightly **bulk CSV export**. **Gotcha we hit and fixed:** the API's *list* endpoint is a capped/rolling window (~2,352 of ~24,000 votes) that silently omits older votes — it cannot be used to enumerate or to find a given file's vote. We now resolve files via the **complete bulk export** and fetch details by id.
- **Committee roll-calls** — there is **no API**; each committee publishes per-meeting results as **PDFs** on europarl.europa.eu, and that page is a **rolling window** (old PDFs age off). Implications: (a) we parse PDFs (brittle by nature; handled defensively), and (b) **the corpus must accumulate over time** — a one-shot scrape cannot rebuild history. The scraper now *merges* rather than overwrites, and the corpora are committed (not treated as regenerable). **Guardrail added 2026-06-09:** `praevisa.corpus_health` is a smoke alarm over the corpora — it fails (non-zero, gating the weekly cadence before any predict/commit) on *shrinkage* (a re-scrape that loses records) or *signal drop-out* (a subject the classifier should recognise but doesn't, the exact bug class fixed that day). Subject parsing is now pinned by `tests/`.
- **Reliability summary:** plenary data is solid and complete via the bulk export; committee data is the fragile, accumulate-over-time dependency and the main data risk — now with a drift guard and parser regression tests in front of it.

6. What you are inheriting (codebase state)

- **Language/stack:** Python; reproducible via `uv`; results are committed JSON + Markdown, regenerated by one command each. Statistics use numpy/scipy. No heavyweight ML framework in the validated path — the baselines and the committee signal are deliberately simple.
- **Validated vs experimental:** the §3 baselines and §4 Stage-A result are committed and reproducible. The forward-prediction product engine (an LLM/Monte-Carlo pipeline that predates this work) is **not** part of what's validated here and should be treated as experimental.
- **Automation just stood up:** a **Stage B** pre-commitment harness writes immutable, hash-stamped predictions for files that have a committee vote but no plenary vote yet, and grades them only once the plenary vote exists *and is dated after the commit*. A weekly macOS **LaunchAgent** runs the full loop (scrape -> health-gate -> grade -> predict -> report -> commit); its first run completed end-to-end. The git commit timestamp is the pre-commitment proof. `praevisa.stage_b report` renders a legible prospective scoreboard (`results/stage_b_report.md`) that leads with the **contested-pending** count.
- **The binding constraint is honest and structural:** there are currently **7 pending predictions but 0 contested** awaiting plenary — the contested files in the rolling window have all already voted. Prospective *contested* evidence (the only kind that upgrades Stage A's p=0.039 from "snapshot" to "track record")

accrues only as new contested committee votes arrive in future scrapes. This is a data-arrival wait, not a code gap; the dashboard surfaces the count so the moment one enters the queue is visible.

- **Caveat to fix for external defensibility:** the repo has **no git remote**, so pre-commitment is currently proven only by *local* history. For an investor- or court-defensible track record, add a remote and push at prediction time. **(Open decision — needs your go-ahead; not done unilaterally since it publishes the IP.)**

7. Competitive benchmark — first measured (2026-06-09)

We tried to put a real number behind "Praevisa wins the contested tail" against the two known rivals. Full artifact: `results/competitor_benchmark_2026-06-08.md`.

- **EU Matrix — structural forfeit.** Its live API is a single frozen snapshot: every predicted group vote is stamped `2024-07-18`, the product is 306 fixed "quizzes" with only 105 historical roll-call links, all pre-July-2024. Our contested files are 2025-26, so a July-2024 model has *zero* predictions for them — there is no overlap to grade. It forfeits the forward contested category by design. Even inside its own backward snapshot it returns "undetermined" most on exactly the pivotal swing groups (NI 19%, ECR 19%, Greens 15%, Renew 12%) — weakest where forecasting value lives. A citable finding.
- **MEPanalytics — the decisive head-to-head, still open.** This is the real threat (forward ML, ~80% claimed accuracy, 2.9M votes). Access is gated ("early access"), so its per-file predictions are not queryable and we **could not benchmark it.** Until we grade their contested-subset predictions against ours, "Praevisa beats MEPanalytics on contested votes" is a **hypothesis, not a result** — do not let it be stated as fact.
- **vs the naive baseline** — Praevisa wins (Stage A committee signal ~3× lower per-group error on contested files, now significant at p=0.039), but that is n=13 (8 contested), one snapshot, not yet prospective.

Takeaway for you: getting MEPanalytics trial access (or their published predictions) is the highest-leverage open item — that one benchmark is the spine of any anti-MEPanalytics pitch and it does not yet exist.

8. Honest limitations and risks

- **Sample is tiny.** Stage A is 13 files (8 contested), one snapshot; the contested files are clustered (shared committees, one near-twin pair) so the *effective* independent contested n is ~3-4. Significant (p=0.039) but encouraging, not conclusive.
- **Not yet truly prospective.** Stage A used historical data with a time-split. The real test (Stage B, blinded pre-commitment) needs ~**2-6 months** to accumulate contested predictions — right now there are *zero* contested files awaiting a plenary vote, because the ones we have already voted.
- **Narrow scope.** The committee signal buys ~weeks of lead time on files that have reached committee. It does **not** forecast far ahead, and it does **not** cover the **Council/trilogue** dimension, where much of the hypothesized commercial value lives. *Feasibility spike done 2026-06-09 (`COUNCIL_LAYER.md`):* real per?member?state Council voting data **exists, is actively maintained (modified 2025?03?10, back to 2009), and is keyed by the same `YYYY/NNNN(COD)` id** — so it joins directly to the 55 procedures we already hold. It is currently **blocked on access** (the Council site is bot?checked; the bulk/SPARQL endpoints 404), which is an engineering gate, not a missing?data gate. The kill?switch is in code (`praevisa.council_feasibility`) and flips to a real triple count the moment a pull works.
- **Competition.** A better-resourced competitor (MEPanalytics) already does forward EP prediction at ~80% accuracy on millions of historical votes, and we have **not** yet benchmarked against it (§7). Our only defensible ground is the contested/upstream niche this signal targets — *if* it validates prospectively *and* it beats MEPanalytics there.

9. Recommendation and where your judgment is needed

1. **Do not make external/marketing/investor claims off this yet.** It is one promising snapshot. Let the automated weekly cadence build a *prospective, blinded* contested track record over the next couple of months; then we either have a defensible edge or we learn the snapshot was lucky — both worth more than asserting it now.
2. **Run the MEPanalytics head-to-head.** Get trial access (or their published predictions) and grade their contested-subset predictions against ours plus the baseline (§7). It is the highest-leverage open item — the number that turns the competitive story from hypothesis into evidence.
3. **Decisions for you:** (a) **Council/trilogue layer** — the feasibility spike de?risked the *data* question (it exists, is maintained, and pairs by procedure; `COUNCIL_LAYER.md`), so the open call is whether to fund the **access?engineering** task (a time?boxed SPARQL/ portal pull, with the VoteWatch 2009-2022 set as a static fallback) that unlocks a real cross?institutional backtest. (b) add a git remote so pre?commitments are externally provable? (c) the committee PDF scraper is now hardened (drift guard + parser tests, §6) — remaining call is how much further to invest in scraper resilience.

10. Get oriented (run these from the `praevisa` repo)

```
uv run python -m praevisa.baseline_eval        # baselines — the real, reproducible numbers
uv run python -m praevisa.baseline_robustness  # significance: Baseline A ties the floor
uv run python -m praevisa.contested_subset     # contested subset indicts Baseline A
uv run python -m praevisa.stage0_feasibility   # data pairs available: 19 (8 contested)
uv run python -m praevisa.stage_a              # committee signal beats floor on contested
(n=13, p=0.039)
uv run python -m praevisa.stage_b status       # prospective prediction ledger
```

Start with `METHODOLOGY.md` and `RESULTS.md` for full method and numbers, and `results/competitor_benchmark_2026-06-08.md` for the competitive read (§7). The work in this briefing is the commit range from `dcf442b` (significance testing) through `7a77f9a` (phantom disambiguation); notable commits: `2dcf53d` (data resolver), `caaddfc` (data feasibility), `0fea9ca` (Stage A result), `649796b` (Stage B harness), `7805292`/`39ff209` (automation), `317293b` (competitor benchmark).